

Testes automatizados em API

Vitor Mattos

Quem sou eu?

Realizador de sonhos desde 2003

Amante de opensource

Palestrante

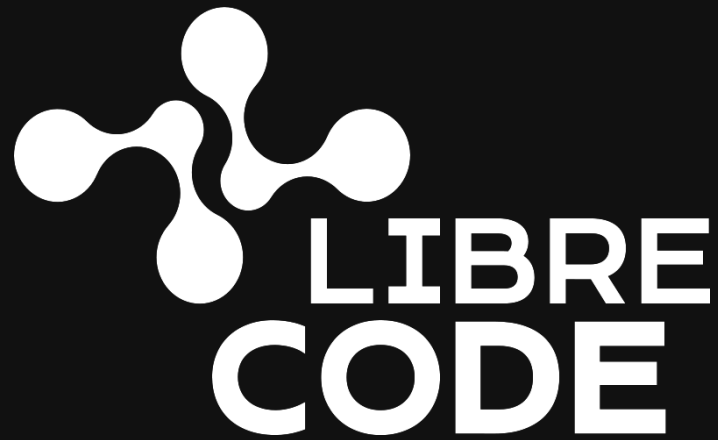
PHP Zend Certified Engineer ([ZEND024235](#))

PHPRio (<https://telegram.me/phprio>)

CTO LibreCode

Redes sociais: (VitorMattos ou VitorMattosRJ)





- Economia solidária
- Segurança e privacidade
- Comunidades de Software Livre
- Transparência (dados abertos)
- Copyleft, claro! LibreCode!



Cronograma

- Conhecendo a API
- Implementando testes
 - estáticos
 - unitários
 - funcionais
 - documentação
- Automatizando tudo

Preparando ambiente



```
git clone https://github.com/vitormattos/api-test  
cd api-test
```

Preparando ambiente

Com docker



```
docker-compose up
```

Preparando ambiente

Sem docker



```
php -S 0.0.0.0:8080 -t public
```

Endpoints

Objetivo: Manipular arquivo storage/database.json

Verbo	Endpoint	Body
GET	/api/user	
POST	/api/user	{"name": "oi", "id": 1}
DELETE	/api/user/{id}	

Análise estática - PHP_CodeSniffer

PSR



```
composer require --dev squizlabs/php_codesniffer
```

https://github.com/squizlabs/PHP_CodeSniffer

Análise estática - PHP_CodeSniffer

PSR



```
vendor/bin/phpcs
```

https://github.com/squizlabs/PHP_CodeSniffer

Análise estática - PHP_CodeSniffer

PSR



```
vendor/bin/phpcs --standard=PSR12 -n src -n public
```

https://github.com/squizlabs/PHP_CodeSniffer

Análise estática - PHP_CodeSniffer

PSR

composer.json

```
"scripts" : {  
    "phpcs" : "phpcs --standard=PSR12 -n src"  
}
```

https://github.com/squizlabs/PHP_CodeSniffer

Análise estática - PHP_CodeSniffer

Config file

.phpcs.xml, phpcs.xml, .phpcs.xml.dist, phpcs.xml.dist

```
1 <?xml version="1.0"?>
2 <ruleset xmlns:xsi="http://www.w3.org/2001/XMLSchema-
  instance" name="PHP_CodeSniffer">
3   <description>The coding standard for PHP_CodeSniffer
  itself.</description>
4   <arg name="colors"/>
5   <file>./src</file>
6   <file>./public</file>
7   <rule ref="PSR12"></rule>
8 </ruleset>
```

Análise estática - psalm

install



```
composer require --dev vimeo/psalm  
./vendor/bin/psalm --init
```

<https://psalm.dev/>

Análise estática - psalm

Verificando



```
./vendor/bin/psalm
```

Verificando com mais detalhes



```
./vendor/bin/psalm --show-info=true
```

<https://psalm.dev/>

Análise estática - psalm

composer.json

```
1  "scripts" : {  
2      "phpcs" : "phpcs",  
3      "phpcbf" : "phpcbf",  
4      "psalm" : "psalm",  
5      "test" : [  
6          "@phpcs",  
7          "@psalm"  
8      ]  
9  }
```

<https://psalm.dev/>

Análise estática - PHPStan

install



```
composer require --dev phpstan/phpstan
```

<https://phpstan.org/>

Análise estática - PHPStan

composer.json

```
1  "scripts" : {  
2      "phpcs" : "phpcs",  
3      "phpcbf" : "phpcbf",  
4      "psalm" : "psalm",  
5      "phpstan" : "phpstan analyse src --level 5",  
6      "test" : [  
7          "@phpcs",  
8          "@psalm",  
9          "@phpstan"  
10     ]  
11 }
```

Levels: 0-9

<https://phpstan.org/>

Análise estática - Phan

Requer extensão, setup mais complexo

<https://github.com/phan/phan>

Análise estática - Phan

Phan in Browser

(Uses [Phan](#) 4.0.0 and PHP 8.0.0. Best viewed in Firefox/Chrome)

Fork me on GitHub

ANALYZE

RUN

```

1 <?php
2 function demo($param, $arg) : ?int {
3     // Phan can infer types, detect missing elements,
4     // and detect incorrect calls.
5     global $argc;
6     var_export(new my_class());
7
8     $cond = $param > 5;
9     if (!($arg instanceof SplObjectStorage)) { return $argc; }
10    $arg->attach($cond);
11    $arg->attach(new stdClass());
12    $arg->attach(
13        new MyClass($argc,
14            [],
15            'extra'
16        );
17    // Phan can detect redundant/impossible conditions
18    // and unused variables.
19    $always_true = is_bool($cond);
20    echo "Missing variable is $argv\n";
21    return $arg;
22 }
23
24 class MyClass {
25     public function __construct(string $x = null) {
26         // and detect undeclared properties, etc.
27         $this->x = $x;
28     }
29 }
30 demo(2, new SplObjectStorage());

```

```

input:6: PhanUndeclaredClassMethod Call to method __construct from
undeclared class \my_class (Did you mean class \MyClass)
input:10: PhanTypeMismatchArgumentInternalReal Argument 1 ($object) is
$cond of type bool but \SplObjectStorage::attach() takes object
input:11: PhanUndeclaredMethod Call to undeclared method
\SplObjectStorage::atach (Did you mean expr->attach())
input:12: PhanParamTooManyInternal Call with 3 arg(s) to
\SplObjectStorage::attach(object $object, $inf = null) which only takes 2
arg(s). This is an ArgumentCountError for internal functions in PHP 8.0+.
input:13: PhanTypeMismatchArgumentInternal Argument 1 ($x) is $argc of type int
but \MyClass::__construct() takes ?string defined at input:25
input:19: PhanRedundantCondition Redundant attempt to cast $cond of type
bool to bool
input:19: PhanUnusedVariable Unused definition of variable $always_true
input:20: PhanTypeSuspiciousStringExpression Suspicious type null= of a
variable or expression $argv used to build a string. (Expected type to be
able to cast to a string)
input:20: PhanUndeclaredVariable Variable $argv is undeclared (Did you
mean $arg or $argc or (global $argv))
input:21: PhanTypeMismatchReturnReal Returning $arg of type
\SplObjectStorage but demo() is declared to return ?int
input:27: PhanUndeclaredProperty Reference to undeclared property
\MyClass->x

```

Unitário - PHPUnit

Tem documentação em português

<https://phpunit.de/>

Unitário - PHPUnit



```
composer require --dev phpunit/phpunit  
mkdir tests
```

<https://phpunit.de/>

Unitário - PHPUnit

tests/unit/Controller/UserControllerTest.php

```
1 <?php declare(strict_types=1);  
2  
3 use PHPUnit\Framework\TestCase;  
4  
5 final class UserControllerTest extends TestCase  
6 {  
7     public function testTrue(): void  
8     {  
9         $this->assertTrue(true);  
10    }  
11 }
```

<https://phpunit.de/>

Unitário - PHPUnit

Configure: **phpunit.xml**

```
1 <testsuites>
2   <testsuite name="default">
3     <directory suffix="Test.php">tests/Unit</directory>
4     <directory suffix="Test.php">tests/Doc</directory>
5   </testsuite>
6   <testsuite name="unit">
7     <directory suffix="Test.php">tests/Unit</directory>
8   </testsuite>
9   <testsuite name="doc">
10    <directory suffix="Test.php">tests/Doc</directory>
11  </testsuite>
12 </testsuites>
```

<https://phpunit.de/>

Unitário - PHPUnit

run



```
vendor/bin/phpunit --testsuite unit
```

<https://phpunit.de/>

Funcional - Behat

install



```
composer require --dev behat/behat  
vendor/bin/behat --init
```

Testando documentação

install



```
composer require --dev byjg/swagger-test
```

<https://github.com/byjg/php-swagger-test>

Testando documentação

run



```
vendor/bin/phpunit --testsuite doc
```

<https://github.com/byjg/php-swagger-test>

Automação de testes - GitHub Actions

<https://docs.github.com/pt/actions>

```
name: Psalm
on: pull_request

jobs:
  psalm:
    runs-on: ubuntu-latest
    strategy:
      fail-fast: false
      matrix:
        php-versions: ['8.1', '8.2']

    name: psalm check php${{ matrix.php-versions }}

    steps:
      - name: Checkout
        uses: actions/checkout@master
```

Conclusão

“Eu cheguei a seguinte conclusão: não adianta consertar o resto, conserta a gente ajuda pra caramba”

Renato Russo

Que país é esse?



Obrigado!

Redes sociais:
(VitorMattos ou VitorMattosRJ)

